

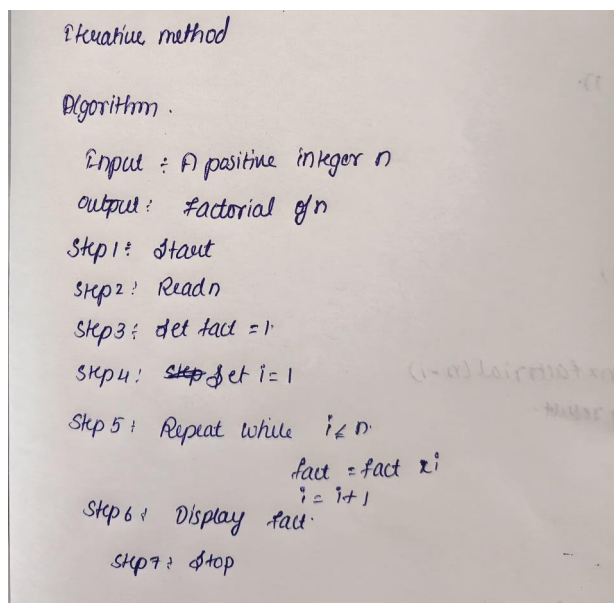


PRACTICAL: 04

Aim: To implement and find the Time analysis of factorial program using iterative and recursive method.

A) Iterative Method

Algorithm:



Program / Code:

```
#include <iostream>

using namespace std;

class Factorial {
public:
    int fact(int n) {
        int result = 1;
        for (int i = 1; i <= n; i++) {
            result = result * i;
        }
        return result;
    }
}
```

```
};  
  
int main() {  
    int n, result;  
    Factorial obj;  
    cout << "Enter the value to find its factorial: ";  
    cin >> n;  
    result = obj.fact(n);  
    cout << "Using Iterative Method!" << endl;  
    cout << "Factorial of " << n << " is " << result << endl;  
    return 0;  
}
```

Output:



```
"C:\Users\karth\Downloads\ID x + v  
Enter the value to find its factorial: 8  
Using Iterative Method!  
Factorial of 8 is 40320  
  
Process returned 0 (0x0)   execution time : 7.026 s  
Press any key to continue.
```

Time Complexity

Time complexity :
for (int i = 1; i <= n; i++) {
 for (int j = 1; j <= w; j++) {
 // ...
 }
}

 $T(n, w) = n \cdot w$
Best case :
even if the items are not suitable program
still fills the complete dp table.
 $O(nw)$
Average case :
the program still fills all ~~n~~ nw entries
 $O(nw)$
Worst case :
the program still fills all nw entries
 $O(nw)$



B) Recursive Method:

Algorithm:

B. Recursive method.

Algorithm :

input : A positive integer n .

output : factorial of n .

Step 1 : Start

Step 2 : Read n .

Step 3 : if $n=0$ or $n=1$

Step 4 : otherwise.

return $n \times \text{factorial}(n-1)$

Step 5 : - Display result.

Step 6 : Stop

Program / Code:

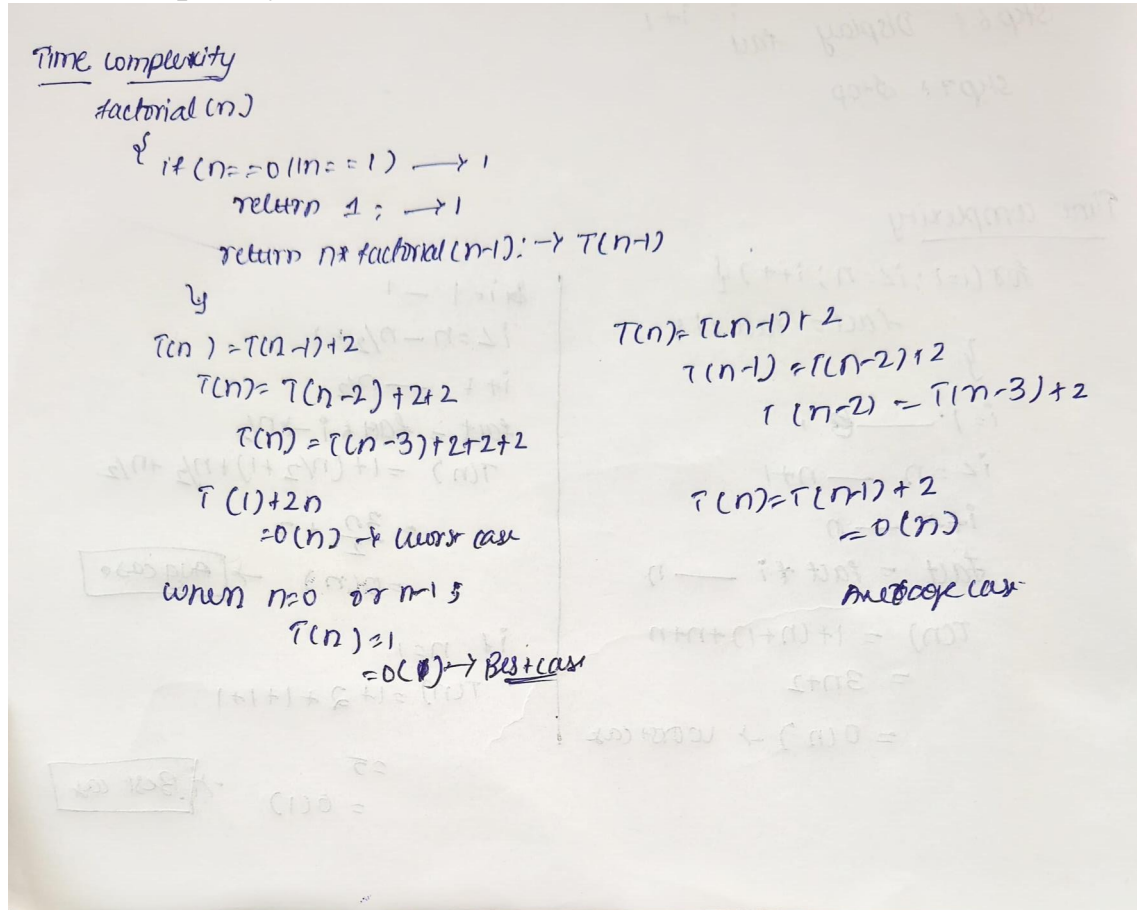
```
#include <iostream>
using namespace std;
class Factorial {
public:
    int factr(int n) {
        if (n == 0 || n == 1) {
            return 1;
```



```
    }  
    else {  
        int result = n * factr(n - 1);  
        return result;  
    }  
}  
};  
  
int main() {  
    int n, result;  
    Factorial obj;  
    cout << "Enter the value to find its factorial: ";  
    cin >> n;  
    result = obj.factr(n);  
    cout << "Using Recursive Method!" << endl;  
    cout << "Factorial of " << n << " is " << result << endl;  
    return 0;  
}
```

Output:

Time Complexity:



Conclusion:

In this practical, we studied and implemented the factorial using both iterative and recursive methods. The iterative method uses a loop, while the recursive method uses function calls. Both methods have $O(n)$ time complexity in the general case. The iterative method uses $O(1)$ space, whereas the recursive method uses $O(n)$ space due to the recursive call stack.